

O3 CPU 模型代码详细分析

概述

`/home/mikasa/0403/gem5/src/cpu/o3` 目录包含完整的 **O3 (Out-of-Order) CPU 模型**，实现了一个复杂的乱序超标量处理器。该模型支持 SMT (Simultaneous Multi-Threading)，最多支持 4 个线程。

本文档基于源代码实现进行详细分析，包括每个类、函数的具体实现细节。

一、核心架构

1.1 主 CPU 类 (cpu.hh / cpu.cc)

类: **CPU** (继承自 **BaseCPU**)

1.1.1 类定义和成员变量

```
class CPU : public BaseCPU
{
public:
    enum Status {
        Running,
        Idle,
        Halted,
        Blocked,
        SwitchedOut
    };

    BaseMMU *mmu;
    using LSQRequest = LSQ::LSQRequest;

    // 状态
    Status _status;

    // 全局序列号
    InstSeqNum globalSeqNum;
    FTSeqNum globalFTSeqNum;

protected:
    // 流水线阶段对象
    BAC bac;           // 分支地址计算
    FTQ ftq;           // 取指目标队列
    Fetch fetch;        // 取指
    Decode decode;      // 译码
    Rename rename;      // 重命名
    IEW iew;           // 发射/执行/写回
    Commit commit;      // 提交

    // 寄存器资源
```

```

PhysRegFile regFile;           // 物理寄存器文件
UnifiedFreeList freeList;      // 空理寄存器空闲列表
PerThreadUnifiedRenameMap renameMap; // 重命名映射
PerThreadUnifiedRenameMap commitRenameMap; // 提交点重命名映射
ROB rob;                       // 重排序缓冲
Scoreboard scoreboard;         // 记分板

// 线程管理
std::list<ThreadID> activeThreads;
std::unordered_map<ThreadID, bool> exitingThreads;
std::vector<ThreadState *> thread;

// 时间缓冲 - 阶段间通信
TimeBuffer<TimeStruct> timeBuffer; // 反向通信
TimeBuffer<FetchStruct> fetchQueue; // 取指->译码
TimeBuffer<DecodeStruct> decodeQueue; // 译码->重命名
TimeBuffer<RenameStruct> renameQueue; // 重命名->IEW
TimeBuffer<IEWStruct> iewQueue; // IEW->提交

public:
// 全局指令列表（用于调试和检查）
std::list<DynInstPtr> instList;
std::queue<ListIt> removeList;
bool removeInstsThisCycle;
};

```

1.1.2 构造函数分析

```

CPU::CPU(const BaseO3CPUParams &params)
: BaseCPU(params),
// 初始化事件
tick tickEvent([this] { tick(); }, "O3CPU tick", false,
Event::CPU_Tick_Pri),
threadExitEvent([this] { exitThreads(); }, "O3CPU exit threads",
false,
Event::CPU_Exit_Pri),

// 初始化流水线阶段
bac(this, params),
ftq(this, params),
fetch(this, params),
decode(this, params),
rename(this, params),
iew(this, params),
commit(this, params),

// 初始化寄存器资源
regFile(params.numPhysIntRegs, params.numPhysFloatRegs,
params.numPhysVecRegs, params.numPhysVecPredRegs,
params.numPhysMatRegs, params.numPhysCCRegs,
params.isa[0]->regClasses()),

```

```

    freeList(name() + ".freelist", &regFile),

    rob(this, params),

    scoreboard(name() + ".scoreboard", regFile.totalNumPhysRegs()),

    // 初始化时间缓冲
    timeBuffer(params.backComSize, params.forwardComSize),
    fetchQueue(params.backComSize, params.forwardComSize),
    decodeQueue(params.backComSize, params.forwardComSize),
    renameQueue(params.backComSize, params.forwardComSize),
    iewQueue(params.backComSize, params.forwardComSize),

    // 初始化活动记录器
    activityRec(name(), NumStages,
                params.backComSize + params.forwardComSize,
    params.activity),

    // 序列号初始化
    globalSeqNum(1),
    globalFTSeqNum(1),

    cpuStats(this)
{
    // 设置阶段指针和相互引用
    // ... 省略详细设置代码
}

```

关键初始化步骤:

1. 创建所有流水线阶段对象
2. 初始化物理寄存器文件和空闲列表
3. 设置时间缓冲，建立阶段间通信
4. 为每个线程创建 ThreadState 和 ThreadContext
5. 初始化重命名映射，分配初始物理寄存器

1.1.3 tick() 函数 - 主循环

```

void CPU::tick()
{
    DPRINTF(O3CPU, "\n\nO3CPU: Ticking main, O3CPU.\n");
    assert(!switchedOut());
    assert(drainState() != DrainState::Drained);

    ++baseStats.numCycles;
    updateCycleCounters(BaseCPU::CPU_STATE_ON);

    // 1. 依次执行每个流水线阶段
    bac.tick();
    fetch.tick();

```

```

    decode.tick();
    rename.tick();
    iew.tick();
    commit.tick();

    // 2. 推进所有时间缓冲
    timeBuffer.advance();
    fetchQueue.advance();
    decodeQueue.advance();
    renameQueue.advance();
    iewQueue.advance();
    activityRec.advance();

    // 3. 清理已移除指令
    if (removeInstsThisCycle) {
        cleanUpRemovedInsts();
    }

    // 4. 调度 CPU 活动
    if (!tickEvent.scheduled()) {
        if (_status == SwitchedOut) {
            lastRunningCycle = curCycle();
        } else if (!activityRec.active() || _status == Idle) {
            lastRunningCycle = curCycle();
            cpuStats.timesIdled++;
        } else {
            schedule(tickEvent, clockEdge(Cycles(1)));
        }
    }

    // 5. 更新线程优先级 (非全系统模式)
    if (!FullSystem)
        updateThreadPriority();

    // 6. 检查 drain 状态
    tryDrain();
}

```

tick() 执行流程:

1. **阶段执行**: 按顺序调用每个阶段的 tick() 函数
2. **缓冲推进**: 所有时间缓冲前进一个周期
3. **指令清理**: 清理已提交或被 squash 的指令
4. **活动调度**: 根据是否有活动调度下一个 tick 事件
5. **优先级更新**: 在 SMT 模式下更新线程优先级
6. **drain 检查**: 检查是否需要排空流水线

1.1.4 寄存器访问函数

物理寄存器访问:

```
// 读取物理寄存器
RegVal getReg(PhysRegIdPtr phys_reg, ThreadID tid);
void getReg(PhysRegIdPtr phys_reg, void *val, ThreadID tid);
void *getWritableReg(PhysRegIdPtr phys_reg, ThreadID tid);

// 写入物理寄存器
void setReg(PhysRegIdPtr phys_reg, RegVal val, ThreadID tid);
void setReg(PhysRegIdPtr phys_reg, const void *val, ThreadID tid);
```

架构寄存器访问 (通过重命名映射):

```
// 读取架构寄存器
RegVal getArchReg(const RegId &reg, ThreadID tid);
void getArchReg(const RegId &reg, void *val, ThreadID tid);
void *getWritableArchReg(const RegId &reg, ThreadID tid);

// 写入架构寄存器
void setArchReg(const RegId &reg, RegVal val, ThreadID tid);
void setArchReg(const RegId &reg, const void *val, ThreadID tid);
```

1.1.5 杂项寄存器访问函数

```
// 无副作用读取
RegVal readMiscRegNoEffect(int misc_reg, ThreadID tid) const;

// 带副作用读取 (触发架构定义的副作用)
RegVal readMiscReg(int misc_reg, ThreadID tid);

// 无副作用设置
void setMiscRegNoEffect(int misc_reg, RegVal val, ThreadID tid);

// 带副作用设置
void setMiscReg(int misc_reg, RegVal val, ThreadID tid);
```

1.1.6 Squash 处理函数

```
// 从 ThreadContext 触发的 squash (外部状态更新)
void squashFromTC(ThreadID tid)
{
    // 停止该线程, 清除未提交指令
}

// 指令管理函数
ListIt addInst(const DynInstPtr &inst); // 添加指令
void instDone(ThreadID tid, const DynInstPtr &inst); // 指令完成
```

```
void removeFrontInst(const DynInstPtr &inst);           // 移除前端指令
void removeInstsNotInROB(ThreadID tid);               // 移除 ROB 外的指令
void removeInstsUntil(const InstSeqNum &seq_num, ThreadID tid); // 移除指定
序列号前的指令
void squashInstIt(const ListIt &instIt, ThreadID tid); // squash 指令
void cleanUpRemovedInsts();                           // 清理已移除指令
```

二、流水线前端

2.1 BAC 阶段 - 分支地址计算 (bac.hh / bac.cc)

类: **BAC**

2.1.1 BAC 架构设计

BAC (Branch and Address Calculation) 阶段负责:

1. **分支预测**: 管理分支预测单元 (BPredUnit)
2. **取指目标生成**: 生成取指目标插入 FTQ
3. **PC 更新**: 计算下一条指令的 PC

BAC 支持两种模式:

- **耦合前端**: BAC 与 Fetch 同步, 每条指令预译码时预测
- **解耦前端**: BAC 独立于 Fetch 运行, 预先生成取指目标

2.1.2 关键数据结构

```
class BAC
{
public:
    enum BACStatus {
        Active,
        Inactive
    };

    enum ThreadStatus {
        Idle,
        Running,
        Squashing,
        Blocked,
        FTQFull,
        FTQLocked,
        ThreadStatusMax
    };

private:
    BACStatus _status;
    ThreadStatus bacStatus[MaxThreads];
```

```

// 分支预测器
branch_prediction::BPredUnit *bpu;

// 取指目标队列
FTQ *ftq;

// 解耦 PC (领先于 fetch)
std::unique_ptr<PCStateBase> bacPC[MaxThreads];

// 配置参数
const bool decoupledFrontEnd; // 是否启用解耦前端
const unsigned fetchTargetWidth; // 取指目标宽度
const unsigned minInstSize; // 最小指令大小
const unsigned maxFTPerCycle; // 每周期最大 FT 数
const unsigned maxTakenPredPerCycle; // 每周期最大 taken 预测数
};

```

2.1.3 tick() 函数

```

void BAC::tick()
{
    bool activity = false;
    bool status_change = false;

    if (decoupledFrontEnd) {
        // 解耦前端模式
        for (const ThreadID tid : *activeThreads) {
            // 检查信号并更新状态
            status_change = status_change || checkSignalsAndUpdate(tid);

            // 生成取指目标
            if (bacStatus[tid] == Running) {
                generateFetchTargets(tid, status_change);
                activity = true;
            }
            stats.status[bacStatus[tid]]++;
        }
    } else {
        // 耦合前端模式 - 只更新分支预测信号
        for (const ThreadID tid : *activeThreads) {
            checkAndUpdateBPUSignals(tid);
            if (bacStatus[tid] != Idle) {
                bacStatus[tid] = Idle;
                status_change = true;
            }
        }
    }

    if (status_change) {
        updateBACStatus();
    }
}

```

```

    }

    if (activity) {
        cpu->activityThisCycle();
    }
}

```

2.1.4 generateFetchTargets() - 解耦前端核心

```

void BAC::generateFetchTargets(ThreadID tid, bool &status_change)
{
    PCStateBase &cur_pc = *bacPC[tid];
    int num_ft = 0;
    int num_taken = 0;

    // 循环生成取指目标
    while ((num_ft < maxFTPerCycle) && (num_taken < maxTakenPredPerCycle))
    {
        Addr search_addr = cur_pc.instAddr();
        Addr start_addr = search_addr;

        // 创建新的取指目标
        FetchTargetPtr curFT = newFetchTarget(tid, cur_pc);
        num_ft++;

        bool branch_found = false;
        bool predict_taken = false;

        // 使用 BTB 搜索分支
        while (true) {
            // 检查 BTB 是否有分支
            branch_found = bpu->BTBValid(tid, search_addr);

            if (branch_found) {
                break;
            }

            // 检查是否达到搜索宽度
            if ((search_addr - start_addr) >= fetchTargetWidth) {
                break;
            }

            search_addr += minInstSize;
        }

        // 更新当前 PC
        cur_pc.set(search_addr);

        std::unique_ptr<PCStateBase> next_pc(cur_pc.clone());
        StaticInstPtr staticInst = nullptr;
    }
}

```



```

    if (branch_found) {
        // 从 BTB 获取分支指令
        staticInst = bpu->BTBGetInst(tid, cur_pc.instAddr());
        assert(staticInst);

        // 进行分支预测
        predict_taken = predict(tid, staticInst, curFT, *next_pc);

        stats.branches++;
        if (predict_taken) {
            stats.predTakenBranches++;
            num_taken++;
        }
    }

    // 设置下一条 PC
    if (!predict_taken) {
        next_pc->set(cur_pc.instAddr() + minInstSize);
    }

    // 完成取指目标
    curFT->finalize(cur_pc, branch_found, predict_taken, *next_pc);
    ftq->insert(tid, curFT);
    wroteToTimeBuffer = true;

    stats.ftSizeDist.sample(search_addr - start_addr);

    // 设置 BAC PC
    set(cur_pc, *next_pc);

    // 检查 FTQ 是否满
    if (ftq->isFull(tid)) {
        bacStatus[tid] = FTQFull;
        status_change = true;
        break;
    }
}
stats.ftNumber.sample(num_ft);
}

```

解耦前端工作原理:

1. 从当前 PC 开始, 使用 BTB 搜索指令流中的分支
2. 遇到分支或达到搜索宽度时停止
3. 如果找到分支, 使用 BPU 预测分支方向
4. 创建取指目标 (类似基本块), 包含分支预测信息
5. 插入 FTQ, 供 Fetch 消费

2.1.5 predict() - 分支预测

```

bool BAC::predict(ThreadID tid, const StaticInstPtr &inst,
                  const FetchTargetPtr &ft, PCStateBase &pc)
{
    assert(ft->bpuHistory == nullptr);
    // 调用 BPU 预测, 预测历史存入 FTQ
    bool taken = bpu->predict(inst, ft->ftNum(), pc, tid, ft.bpuHistory);

    DPRINTF(Branch, "[tid:%i, ftn:%llu] History added.\n", tid, ft-
>ftNum());
    return taken;
}

```

2.1.6 updatePC() - PC 更新

```

bool BAC::updatePC(const DynInstPtr &inst, PCStateBase &fetch_pc,
                  FetchTargetPtr &ft)
{
    bool predict_taken;
    ThreadID tid = inst->threadNumber;

    if (inst->isControl()) {
        // 控制指令
        if (decoupledFrontEnd) {
            // 解耦模式 - 从 FTQ 读取预测
            predict_taken = updatePreDecode(tid, inst->seqNum,
                                           inst->staticInst, fetch_pc,
ft);
        } else {
            // 耦合模式 - 在此处预测
            predict_taken =
                bpu->predict(inst->staticInst, inst->seqNum, fetch_pc,
tid);
        }

        inst->setPredTarg(fetch_pc);
        inst->setPredTaken(predict_taken);

        ++stats.branches;
        if (predict_taken) {
            ++stats.predTakenBranches;
        }
    } else {
        // 非控指令 - 直接推进 PC
        inst->staticInst->advancePC(fetch_pc);
        inst->setPredTarg(fetch_pc);
        inst->setPredTaken(false);
        predict_taken = false;
    }

    if (decoupledFrontEnd) {

```

```

        // 解耦模式 - 检查是否到达取指目标结尾
        if ((ft->isExitInst(inst->pcState().instAddr()) &&
            (!inst->isMicroop() || inst->isLastMicroop())) ||
            !ftq->isReady(tid)) {
            ft = nullptr;
        }
    }

    return predict_taken;
}

```

2.1.7 BPU 信号更新

```

bool BAC::checkAndUpdateBPUSignals(ThreadID tid)
{
    // 检查来自提交的 squash 信号
    if (fromCommit->commitInfo[tid].squash) {
        // Squash FTQ 和 BPU 历史
        squashBpuHistories(tid);
        squash(*fromCommit->commitInfo[tid].pc, tid);

        // 更新分支预测器
        if (fromCommit->commitInfo[tid].mispredictInst &&
            fromCommit->commitInfo[tid].mispredictInst->isControl()) {
            bpu->squash(fromCommit->commitInfo[tid].doneSeqNum,
                        *fromCommit->commitInfo[tid].pc,
                        fromCommit->commitInfo[tid].branchTaken, tid,
true);

            stats.branchMisspredict++;
            stats.squashBranchCommit++;
        } else {
            bpu->squash(fromCommit->commitInfo[tid].doneSeqNum, tid);
            stats.noBranchMisspredict++;
        }
        return true;
    }

    // 检查来自提交的正常完成
    else if (fromCommit->commitInfo[tid].doneSeqNum) {
        bpu->update(fromCommit->commitInfo[tid].doneSeqNum, tid);
    }

    // 检查来自译码的 squash 信号
    if (fromDecode->decodeInfo[tid].squash) {
        squashBpuHistories(tid);
        squash(*fromDecode->decodeInfo[tid].nextPC, tid);

        if (fromDecode->decodeInfo[tid].branchMispredict) {
            bpu->squash(fromDecode->decodeInfo[tid].doneSeqNum,
                        *fromDecode->decodeInfo[tid].nextPC,
                        fromDecode->decodeInfo[tid].branchTaken, tid,

```

```

false);
    stats.branchMisspredict++;
    stats.squashBranchDecode++;
} else {
    bpu->squash(fromDecode->decodeInfo[tid].doneSeqNum, tid);
    stats.noBranchMisspredict++;
}
return true;
}

// 检查来自取指的 squash 信号
if (fromFetch->fetchInfo[tid].squash && bacStatus[tid] != Squashing) {
    squashBpuHistories(tid);
    squash(*fromFetch->fetchInfo[tid].nextPC, tid);
    return true;
}
return false;
}

```

2.2 FTQ - 取指目标队列 (ftq.hh / ftq.cc)

类: FTQ

FTQ (Fetch Target Queue) 是 BAC 和 Fetch 之间的缓冲队列，存储取指目标。

2.2.1 FetchTarget 结构

```

class FetchTarget
{
public:
    FetchTarget(ThreadID _tid, const PCStateBase &_start_pc,
                FTSeqNum _ft_num)
        : tid(_tid), startPC(_start_pc.clone()), ftNum(_ft_num)
    {
        // 初始化成员
    }

    // 初始化取指目标
    void finalize(const PCStateBase &_end_pc, bool _found_branch,
                 bool _pred_taken, const PCStateBase &_next_pc);

    // 查询函数
    FTSeqNum ftNum() const { return ftNum; }
    ThreadID tid() const { return tid; }
    Addr startAddress() const { return startPC->instAddr(); }
    Addr endAddress() const { return endPC->instAddr(); }
    bool isExitBranch(Addr pc) const;
    bool inRange(Addr pc) const;

    // 预测历史

```

```

    BPredUnit::PredictorHistory *bpuHistory;

    // 读取预测目标
    const PCStateBase& readPredTarg() const { return predTarg; }

private:
    ThreadID tid;
    std::unique_ptr<PCStateBase> startPC;
    std::unique_ptr<PCStateBase> endPC;
    std::unique_ptr<PCStateBase> predTarg;
    FTSeqNum ftNum;
    bool foundBranch;
    bool predTaken;
};

```

2.2.2 FTQ 操作

```

class FTQ
{
public:
    // 插入取指目标
    void insert(ThreadID tid, FetchTargetPtr ft);

    // 读取队首
    FetchTargetPtr readHead(ThreadID tid);

    // 弹出队首
    void popHead(ThreadID tid);

    // Squash
    void squash(ThreadID tid);

    // 状态查询
    bool isEmpty() const;
    bool isEmpty(ThreadID tid) const;
    bool isFull(ThreadID tid) const;
    bool isReady(ThreadID tid) const;
    bool isLocked(ThreadID tid) const;

    // 锁定和解锁
    void lock(ThreadID tid);
    void unlock(ThreadID tid);
    void resetState(ThreadID tid);

    // 反向遍历
    void forAllBackward(ThreadID tid,
        std::function<void(FetchTargetPtr&)> func);
};

```

2.3 Fetch 阶段 - 取指 (fetch.hh / fetch.cc)

类: Fetch

2.3.1 Fetch 类结构

```
class Fetch
{
public:
    enum FetchStatus {
        Active,
        Inactive
    };

    enum ThreadStatus {
        Running,
        Idle,
        Squashing,
        Blocked,
        Fetching,
        TrapPending,
        QuiescePending,
        ItlbWait,
        IcacheWaitResponse,
        IcacheWaitRetry,
        IcacheAccessComplete,
        FtqWait,
        NoGoodAddr,
        ThreadStatusMax
    };

private:
    FetchStatus _status;
    ThreadStatus fetchStatus[MaxThreads];

    // SMT 取指策略
    SMTFetchPolicy fetchPolicy;
    std::list<ThreadID> priorityList;

    // 取指端口
    class IcachePort : public RequestPort
    {
    protected:
        Fetch *fetch;
    public:
        IcachePort(Fetch *_fetch, CPU *_cpu);
        virtual bool recvTimingResp(PacketPtr pkt);
        virtual void recvReqRetry();
    };

    // 翻译端口
    class FetchTranslation : public BaseMMU::Translation
```

```

{
    protected:
        Fetch *fetch;
    public:
        FetchTranslation(Fetch *_fetch) : fetch(_fetch) {}
        void markDelayed() {}
        void finish(const Fault &fault, const RequestPtr &req,
                    gem5::ThreadContext *tc, BaseMMU::Mode mode);
};

// PC 和缓冲区管理
std::unique_ptr<PCStateBase> pc[MaxThreads];
Addr fetchOffset[MaxThreads];
StaticInstPtr macroop[MaxThreads];
bool delayedCommit[MaxThreads];

RequestPtr memReq[MaxThreads];

// 取指缓冲区
uint8_t *fetchBuffer[MaxThreads];
Addr fetchBufferPC[MaxThreads];
bool fetchBufferValid[MaxThreads];
unsigned fetchBufferSize;
Addr fetchBufferMask;

// 取指队列
std::deque<DynInstPtr> fetchQueue[MaxThreads];
unsigned fetchQueueSize;

// 状态标志
bool wroteToTimeBuffer;
int numInst;
bool cacheBlocked;
PacketPtr retryPkt;
ThreadID retryTid;

// 停顿源
struct Stalls {
    bool decode;
    bool drain;
};
Stalls stalls[MaxThreads];

// SMT 相关
ThreadID numThreads;
ThreadID numFetchingThreads;
ThreadID threadFetched;
bool interruptPending;

// 配置参数
const bool decoupledFrontEnd;
const Cycles decodeToFetchDelay;
const Cycles renameToFetchDelay;
const Cycles iewToFetchDelay;

```

```

const Cycles commitToFetchDelay;
const unsigned fetchWidth;
const unsigned decodeWidth;
const unsigned maxFTPerCycle;
const unsigned maxTakenPredPerCycle;

// BAC 和 FTQ 指针
BAC *bac;
FTQ *ftq;
};

```

2.3.2 tick() 函数

```

void Fetch::tick()
{
    bool activity = false;
    bool status_change = false;

    for (const ThreadID tid : *activeThreads) {
        // 检查信号并更新状态
        status_change = status_change || checkSignalsAndUpdate(tid);

        // 取指
        fetch(status_change);
    }

    // 更新总体状态
    if (status_change) {
        updateFetchStatus();
    }

    // 标记活动
    if (wroteToTimeBuffer) {
        activity = true;
        wroteToTimeBuffer = false;
    }

    if (activity) {
        DPRINTF(Activity, "Activity this cycle.\n");
        cpu->activityThisCycle();
    }
}

```

2.3.3 fetch() 函数 - 核心取指逻辑

```

void Fetch::fetch(bool &status_change)
{
    // 检查是否应该取指

```



```

    if (fetchStatus[tid] == Running) {
        DPRINTF(Fetch, "[tid:%i] Fetching instructions.\n", tid);

        // 重置指令计数器
        numInst = 0;

        // 开始取指循环
        while (numInst < fetchWidth) {
            // 解耦前端模式 - 从 FTQ 读取取指目标
            if (decoupledFrontEnd) {
                FetchTargetPtr ft = ftq->readHead(tid);
                if (!ft || !ftq->isReady(tid)) {
                    DPRINTF(Fetch, "[tid:%i] FTQ is empty or locked.\n",
tid);
                        break;
                    }
                }

            // 检查 TLB 是否就绪
            if (memReq[tid] || (fetchBufferValid[tid] &&
                (fetchOffset[tid] >= fetchBufferSize || memReq[tid])))
        {
            if (memReq[tid]) {
                DPRINTF(Fetch, "[tid:%i] Waiting for I-cache
response.\n", tid);
                // 检查是否可以使用流水化取指
                if (issuePipelinedIfetch[tid]) {
                    pipelineIcacheAccesses(tid);
                }
                break;
            }
        }

        // 获取当前 PC
        TheISA::PCState this_pc = *pc[tid];

        // 从取指缓冲区读取指令
        if (!fetchBufferValid[tid]) {
            // 需要从 I-cache 取指
            if (fetchCacheLine(this_pc.instAddr(), tid,
this_pc.instAddr())) {
                // 如果成功取指, 继续处理
            } else {
                // 取指失败, break
                break;
            }
        }

        // 预译码指令
        InstDecoder *decoder = this->decoder[tid];
        Addr fetch_PC = this_pc.instAddr();
        MicroPC micro_pc = this_pc.microPC();

        StaticInstPtr inst = nullptr;

```

```

        if (fetchBufferValid[tid]) {
            // 从取指缓冲区预译码
            uint8_t *inst_data = &fetchBuffer[tid][fetchOffset[tid]];
            inst = decoder->decodeInst(
                fetch_pc, micro_pc, inst_data, fetchBufferSize -
fetchOffset[tid]);
        }

        // 检查预译码结果
        if (inst != nullptr) {
            // 创建动态指令
            DynInstPtr dyn_inst =
                buildInst(tid, inst, macroop[tid], this_pc, this_pc,
false);

            // 更新 PC
            if (decoupledFrontEnd) {
                // 解耦模式 - 使用 BAC 更新 PC
                if (bac->updatePC(dyn_inst, *pc[tid], ftq-
>readHead(tid))) {
                    // 分支 taken, 需要更新 FTQ
                    ftq->popHead(tid);
                }

                // 检查是否完成取指目标
                if (!ftq->readHead(tid) ||
                    ftq->readHead(tid)->isExitInst(pc[tid]-
>instAddr())) {
                    ftq->popHead(tid);
                }
            } else {
                // 耦合模式 - 直接推进 PC
                inst->advancePC(*pc[tid]);
            }

            // 添加到取指队列
            fetchQueue[tid].push_back(dyn_inst);
            wroteToTimeBuffer = true;
            numInst++;

            // 更新取指缓冲区偏移
            fetchOffset[tid] += inst->size();
            macroop[tid] = nullptr;

            // 检查取指缓冲区是否用完
            if (fetchOffset[tid] >= fetchBufferSize ||
                fetchOffset[tid] >= cacheBlkSize) {
                fetchBufferValid[tid] = false;
            }

            // 如果是宏操作, 设置宏操作指针
            if (inst->isMacroop()) {
                macroop[tid] = inst;
            }
        }
    }
}

```

```

        DPRINTF(Fetch, "[tid:%i] Decode: %s\n", tid, *inst);
    } else {
        // 预译码失败
        DPRINTF(Fetch, "[tid:%i] Unknown instruction at PC %#x\n",
            tid, fetch_pc);

        if (memReq[tid]) {
            // 正在等待 I-cache 响应
            DPRINTF(Fetch, "[tid:%i] Waiting for I-cache
response.\n", tid);
            break;
        }

        // 尝试取指下一行
        fetchBufferValid[tid] = false;
        if (fetchCacheLine(this_pc.instAddr(), tid,
this_pc.instAddr())) {
            // 继续
        } else {
            // 没有更多指令
            break;
        }
    }
}

// 写入时间缓冲
if (!fetchQueue[tid].empty()) {
    toDecode->fetchInfo[tid].size = fetchQueue[tid].size();
    toDecode->fetchInfo[tid].tid = tid;

    for (int i = 0; i < fetchQueue[tid].size(); i++) {
        toDecode->fetchInfo[tid].insts[i] = fetchQueue[tid][i];
    }

    wroteToTimeBuffer = true;
}

// 清理取指队列
fetchQueue[tid].clear();

// 如果本周期没有取指任何指令，可能是因为 FTQ 为空
if (numInst == 0 && decoupledFrontEnd && fetchStatus[tid] ==
Running) {
    if (!ftq->isReady(tid)) {
        fetchStatus[tid] = FtqWait;
        status_change = true;
    }
}
}
}

```

2.3.4 fetchCacheLine() - I-cache 取指

```
bool Fetch::fetchCacheLine(Addr vaddr, ThreadID tid, Addr pc)
{
    // 创建内存请求
    RequestPtr req = std::make_shared<Request>();
    req->setVirt(vaddr, cacheBlkSize, Request::INST_FETCH, 0,
               Request::funcPCInst, pc);

    // 创建取指端口
    PortProxy *icache_port = &icachePort;

    // 创建 TLB 转换对象
    FetchTranslation *translation = new FetchTranslation(this);
    translation->start(cpu->tcBase(tid), functional,
                     req, icache_port, cpu->mmu->dtb, Cycles(0));

    memReq[tid] = req;
    fetchOffset[tid] = vaddr - fetchBufferAlignPC(vaddr);
    fetchBufferPC[tid] = vaddr;

    return true;
}

void Fetch::finishTranslation(const Fault &fault, const RequestPtr
&mem_req)
{
    ThreadID tid = mem_req->requestorId();

    if (fault != NoFault) {
        // TLB 错误
        if (fault == pageFault()) {
            // 页面错误 - 需要等待
            fetchStatus[tid] = ItlbWait;
        } else {
            // 其他错误 - 产生 trap
            cpu->trap(fault, tid, nullptr);
        }
        return;
    }

    // 发送 I-cache 请求
    bool success = icachePort.sendTimingReq(mem_req);

    if (!success) {
        // I-cache 繁忙, 需要重试
        if (!retryPkt || retryTid == tid) {
            retryPkt = mem_req->pkt;
            retryTid = tid;
            fetchStatus[tid] = IcacheWaitRetry;
        }
    } else {
```

```

        // 请求成功，清理重试信息
        if (retryPkt && retryTid == tid) {
            retryPkt = nullptr;
        }
        fetchStatus[tid] = IcacheWaitResponse;
    }
}

```

2.3.5 SMT 取指策略

```

ThreadID Fetch::getFetchingThread()
{
    switch (fetchPolicy) {
        case SMTFetchPolicy::RoundRobin:
            return roundRobin();
        case SMTFetchPolicy::IQCount:
            return iqCount();
        case SMTFetchPolicy::LSQCount:
            return lsqCount();
        case SMTFetchPolicy::BranchCount:
            return branchCount();
        default:
            return 0;
    }
}

ThreadID Fetch::roundRobin()
{
    ThreadID tid;
    int count = 0;

    // 轮询优先级列表
    for (auto it = priorityList.begin(); it != priorityList.end(); ++it) {
        if (fetchStatus[*it] == Running) {
            tid = *it;
            // 循环移动到列表末尾
            priorityList.erase(it);
            priorityList.push_back(tid);
            return tid;
        }
        count++;
    }

    // 没有可取指线程
    return -1;
}

```

2.4 Decode 阶段 - 译码 (decode.hh / decode.cc)

类: Decode

2.4.1 Decode 类结构

```
class Decode
{
public:
    enum DecodeStatus {
        Active,
        Inactive
    };

    enum ThreadStatus {
        Running,
        Idle,
        StartSquash,
        Squashing,
        Blocked,
        Unblocking,
        ThreadStatusMax
    };

private:
    DecodeStatus _status;
    ThreadStatus decodeStatus[MaxThreads];

    // 指令队列
    std::queue<DynInstPtr> insts[MaxThreads];

    // 滑移缓冲区
    std::queue<DynInstPtr> skidBuffer[MaxThreads];

    // 停顿源
    struct Stalls {
        bool rename;
    };
    Stalls stalls[MaxThreads];

    // SMT 相关
    ThreadID numThreads;
    std::list<ThreadID> *activeThreads;

    // 配置参数
    Cycles renameToDecodeDelay;
    Cycles iewToDecodeDelay;
    Cycles commitToDecodeDelay;
    Cycles fetchToDecodeDelay;
    unsigned decodeWidth;
    unsigned skidBufferMax;
};
```

2.4.2 tick() 函数

```
void Decode::tick()
{
    bool status_change = false;

    for (const ThreadID tid : *activeThreads) {
        // 检查信号并更新状态
        status_change = status_change || checkSignalsAndUpdate(tid);

        // 执行译码
        decode(status_change, tid);
    }

    // 推进时间缓冲
    toDecode->advance();

    // 更新状态
    if (status_change) {
        updateStatus();
    }
}
```

2.4.3 decode() 函数

```
void Decode::decode(bool &status_change, ThreadID tid)
{
    bool status_local_change = false;

    if (decodeStatus[tid] == Running) {
        // 执行译码
        decodeInsts(tid);
    } else if (decodeStatus[tid] == Unblocking) {
        // 解阻塞 - 处理滑移缓冲区
        if (skidsEmpty()) {
            DPRINTF(Decode, "[tid:%i] Done unblocking, switching to
running.\n", tid);
            decodeStatus[tid] = Running;

            toDecode->decodeInfo[tid].used = true;
            wroteToTimeBuffer = true;
            status_local_change = true;
        }
    } else if (decodeStatus[tid] == Squashing) {
        // Squashing - 检查是否完成
        if (squashInst[tid] == nullptr) {
            DPRINTF(Decode, "[tid:%i] Done squashing, switching to
running.\n", tid);
            decodeStatus[tid] = Running;
            status_local_change = true;
        }
    }
}
```

```

    }

    if (status_local_change) {
        status_change = true;
    }
}

```

2.4.4 decodeInsts() 函数

```

void Decode::decodeInsts(ThreadID tid)
{
    // 获取可用的指令数
    unsigned from_fetch =
        std::min(fetchQueue->fromFetch[tid].size, decodeWidth);

    // 处理所有可用的指令
    for (int i = 0; i < from_fetch; i++) {
        DynInstPtr inst = fetchQueue->fromFetch[tid].insts[i];

        // 检查 PC 相对分支
        if (inst->isPCRelativeBranch()) {
            // 验证 PC 相对分支预测
            if (inst->predPC() != inst->pcState().nextPC()) {
                // 预测错误
                squash(inst, true, tid);
                return;
            }
        }

        // 检查是否是控制指令但预测为非控制
        if (inst->isControl() != inst->staticInst->isControl()) {
            squash(inst, false, tid);
            return;
        }

        // 添加到指令列表
        insts[tid].push(inst);

        DPRINTF(Decode, "[tid:%i] Instruction %i added to list.\n", tid,
i);
    }

    // 发送到重命名
    if (!insts[tid].empty()) {
        toRename->decodeInfo[tid].size = insts[tid].size();
        toRename->decodeInfo[tid].tid = tid;

        int i = 0;
        while (!insts[tid].empty()) {
            toRename->decodeInfo[tid].insts[i] = insts[tid].front();

```



```
        insts[tid].pop();
        i++;
    }

    wroteToTimeBuffer = true;
    DPRINTF(Decode, "[tid:%i] %i instructions forwarded to rename.\n",
tid, i);
}
}
```

三、流水线后端

3.1 Rename 阶段 - 重命名 (rename.hh / rename.cc)

类: **Rename**

3.1.1 Rename 类结构

```
class Rename
{
public:
    enum RenameStatus {
        Active,
        Inactive
    };

    enum ThreadStatus {
        Running,
        Idle,
        StartSquash,
        Squashing,
        Blocked,
        Unblocking,
        SerializeStall,
        ThreadStatusMax
    };

    // 重命名历史
    struct RenameHistory {
        InstSeqNum instSeqNum;
        RegId archReg;
        PhysRegIdPtr newPhysReg;
        PhysRegIdPtr prevPhysReg;
    };

private:
    RenameStatus _status;
    ThreadStatus renameStatus[MaxThreads];

    // 指令队列
```

```

typedef std::deque<DynInstPtr> InstQueue;
InstQueue insts[MaxThreads];
InstQueue skidBuffer[MaxThreads];

// 重命名历史缓冲区
std::list<RenameHistory> historyBuffer[MaxThreads];

// 资源管理
UnifiedRenameMap *renameMap[MaxThreads];
UnifiedFreeList *freeList;
Scoreboard *scoreboard;

// 暂停源
struct Stalls {
    bool iew;
    bool commit;
};
Stalls stalls[MaxThreads];

// 资源追踪
struct FreeEntries {
    unsigned iqEntries;
    unsigned robEntries;
    unsigned lqEntries;
    unsigned sqEntries;
};
FreeEntries freeEntries[MaxThreads];

// 序列化支持
DynInstPtr serializeInst[MaxThreads];
bool serializeOnNextInst[MaxThreads];

// 配置参数
int iewToRenameDelay;
int decodeToRenameDelay;
unsigned commitToRenameDelay;
unsigned renameWidth;
unsigned skidBufferMax;
};

```

3.1.2 tick() 函数

```

void Rename::tick()
{
    bool status_change = false;

    for (const ThreadID tid : *activeThreads) {
        // 检查信号并更新状态
        status_change = status_change || checkSignalsAndUpdate(tid);

        // 执行重命名
    }
}

```

```

        rename(status_change, tid);
    }

    // 更新状态
    if (status_change) {
        updateStatus();
    }
}

```

3.1.3 rename() 函数

```

void Rename::rename(bool &status_change, ThreadID tid)
{
    bool status_local_change = false;

    if (renameStatus[tid] == Running) {
        // 执行重命名
        renameInsts(tid);
    } else if (renameStatus[tid] == Unblocking) {
        // 解阻塞
        if (skidBuffer[tid].empty()) {
            DPRINTF(Rename, "[tid:%i] Done unblocking, switching to
running.\n", tid);
            renameStatus[tid] = Running;
            status_local_change = true;
        }
    } else if (renameStatus[tid] == SerializeStall) {
        // 序列化停顿 - 等待 ROB 空
        if (rob->isEmpty(tid)) {
            DPRINTF(Rename, "[tid:%i] ROB is empty, unblocking.\n", tid);
            renameStatus[tid] = Unblocking;
            status_local_change = true;
        }
    } else if (renameStatus[tid] == Squashing) {
        // Squashing
        if (squashInst[tid] == nullptr) {
            DPRINTF(Rename, "[tid:%i] Done squashing, switching to
running.\n", tid);
            renameStatus[tid] = Running;
            status_local_change = true;
        }
    }

    if (status_local_change) {
        status_change = true;
    }
}

```

3.1.4 renameInsts() 函数 - 核心重命名逻辑

```

void Rename::renameInsts(ThreadID tid)
{
    // 获取可用指令数
    unsigned num_insts = std::min(
        static_cast<unsigned>(insts[tid].size()),
        std::min(renameWidth,
            static_cast<unsigned>(
                freeEntries[tid].iqEntries)));

    // 处理所有可用指令
    for (int i = 0; i < num_insts; i++) {
        DynInstPtr inst = insts[tid].front();
        insts[tid].pop_front();

        // 检查是否有足够的资源
        if (freeEntries[tid].robEntries == 0 ||
            freeEntries[tid].iqEntries == 0 ||
            freeEntries[tid].lqEntries == 0 ||
            freeEntries[tid].sqEntries == 0) {
            // 资源不足, 放回指令队列
            insts[tid].push_front(inst);
            break;
        }

        // 序列化检查
        if (inst->isSerializeBefore()) {
            serializeAfter(insts[tid], tid);
        }

        // 检查是否需要序列化
        if (serializeInst[tid] != nullptr) {
            if (inst->seqNum == serializeInst[tid]->seqNum + 1) {
                // 找到序列化后的第一条指令
                serializeInst[tid] = nullptr;
                renameStatus[tid] = Unblocking;
            } else {
                // 还没找到正确的指令
                insts[tid].push_front(inst);
                break;
            }
        }
    }

    // 重命名目的寄存器
    if (inst->isFirstMicroop() || inst->isLastMicroop() ||
        (!inst->isMicroop() && inst->hasDestReg())) {
        renameDestRegs(inst, tid);
    }

    // 重命名源寄存器
    if (!inst->isNoop() && inst->hasSrcReg()) {
        renameSrcRegs(inst, tid);
    }
}

```

```

// 更新资源计数
if (inst->isLoad()) {
    freeEntries[tid].lqEntries--;
} else if (inst->isStore()) {
    freeEntries[tid].sqEntries--;
} else if (!inst->isNoop() && !inst->isMemBarrier()) {
    freeEntries[tid].iqEntries--;
}

freeEntries[tid].robEntries--;

// 标记指令已重命名
inst->setCanCommit();
DPRINTF(Rename, "[tid:%i] Renamed instruction [sn:%lli].\n",
        tid, inst->seqNum);

// 发送到 IEW
toIEW->renameInfo[tid].insts[toIEWIndex] = inst;
toIEW->renameInfo[tid].size++;
wroteToTimeBuffer = true;

// 更新索引
toIEWIndex++;
if (toIEWIndex >= renameWidth) {
    toIEWIndex = 0;
}

instsInProgress[tid]++;
if (inst->isLoad()) {
    loadsInProgress[tid]++;
} else if (inst->isStore()) {
    storesInProgress[tid]++;
}
}
}

```

3.1.5 renameDestRegs() - 目的寄存器重命名

```

void Rename::renameDestRegs(const DynInstPtr &inst, ThreadID tid)
{
    // 获取目的寄存器列表
    int num_dest_regs = inst->numDestRegs();

    for (int i = 0; i < num_dest_regs; i++) {
        RegId src_reg = inst->destRegIdx(i);
        PhysRegIdPtr phys_reg;
        PhysRegIdPtr prev_phys_reg;

        // 跳过物理寄存器
        phys_reg = freeList->getReg(src_reg.classType());
    }
}

```

```

        // 查找当前映射
        prev_phys_reg = renameMap[tid].lookup(src_reg);

        // 更新映射
        renameMap[tid].setEntry(src_reg, phys_reg);

        // 标记寄存器为未就绪
        scoreboard->unsetReg(phys_reg);

        // 记录重命名历史
        RenameHistory history_entry(inst->seqNum, src_reg, phys_reg,
                                    prev_phys_reg);
        historyBuffer[tid].push_back(history_entry);

        // 设置指令的物理寄存器
        inst->setDestReg(i, phys_reg);

        DPRINTF(Rename, "[tid:%i] Renamed arch reg %i to physical reg
%i\n",
                tid, src_reg.index(), phys_reg->index());
    }

    stats.renamedOperands += num_dest_regs;
}

```

3.1.6 renameSrcRegs() - 源寄存器重命名

```

void Rename::renameSrcRegs(const DynInstPtr &inst, ThreadID tid)
{
    int num_src_regs = inst->numSrcRegs();

    for (int i = 0; i < num_src_regs; i++) {
        RegId src_reg = inst->srcRegIdx(i);
        PhysRegIdPtr phys_reg;

        // 查找当前映射
        phys_reg = renameMap[tid].lookup(src_reg);

        // 设置指令的源寄存器
        inst->setSrcReg(i, phys_reg);

        DPRINTF(Rename, "[tid:%i] Source reg %i mapped to physical reg
%i\n",
                tid, src_reg.index(), phys_reg->index());
    }

    stats.lookups += num_src_regs;
    if (inst->isIntAlk(src_reg.classType())) {
        stats.intLookups += num_src_regs;
    } else if (inst->isFloatAlk(src_reg.classType())) {
        stats.fpLookups += num_src_regs;
    }
}

```

```

    }
}

```

3.1.7 squash() 函数

```

void Rename::squash(const InstSeqNum &squash_seq_num, ThreadID tid)
{
    DPRINTF(Rename, "[tid:%i] Squashing until sequence number %i.\n",
            tid, squash_seq_num);

    // 标记 squash 指令
    if (squashInst[tid] == nullptr) {
        squashInst[tid] = insts[tid].back();
    }

    // 将未处理的指令移到滑动缓冲区
    while (!insts[tid].empty()) {
        skidBuffer[tid].push_back(insts[tid].front());
        insts[tid].pop_front();
    }

    // 设置状态为 Squashing
    renameStatus[tid] = Squashing;
    wroteToTimeBuffer = true;
}

void Rename::doSquash(const InstSeqNum &squash_seq_num, ThreadID tid)
{
    bool done = true;

    // 恢复重命名历史
    while (!historyBuffer[tid].empty()) {
        RenameHistory &history_entry = historyBuffer[tid].back();

        if (history_entry.instSeqNum > squash_seq_num) {
            // 将物理寄存器返回空闲列表
            freeList->addReg(history_entry.archReg.classType(),
                            history_entry.newPhysReg);

            // 恢复映射
            renameMap[tid].setEntry(history_entry.archReg,
                                    history_entry.prevPhysReg);

            historyBuffer[tid].pop_back();
        } else {
            done = false;
            break;
        }
    }

    if (done) {

```

```
        squashInst[tid] = nullptr;
    }
}
```

3.2 IEW 阶段 - 发射/执行/写回 (iew.hh / iew.cc)

类: **IEW**

3.2.1 IEW 类结构

```
class IEW
{
public:
    enum Status {
        Active,
        Inactive
    };

    enum StageStatus {
        Running,
        Blocked,
        Idle,
        StartSquash,
        Squashing,
        Unblocking,
        ThreadStatusMax
    };

private:
    Status _status;
    StageStatus dispatchStatus[MaxThreads];
    StageStatus exeStatus;
    StageStatus wbStatus;

    // 指令队列
    std::queue<DynInstPtr> insts[MaxThreads];
    std::queue<DynInstPtr> skidBuffer[MaxThreads];

    // 资源
    InstructionQueue instQueue;
    LSQ ldstQueue;
    Scoreboard *scoreboard;
    std::vector<FUPool *> fuPools;

    // 状态标志
    bool wroteToTimeBuffer;
    bool updateLSQNextCycle;
    bool updatedQueues;
    bool fetchRedirect[MaxThreads];
}
```



```
// 配置参数
Cycles commitToIEWDelay;
Cycles renameToIEWDelay;
Cycles issueToExecuteDelay;
unsigned dispatchWidth;
unsigned issueWidth;
unsigned wbWidth;
unsigned skidBufferMax;

};
```

3.2.2 tick() 函数

```
void IEW::tick()
{
    bool status_change = false;

    // 1. 检查信号并更新状态
    for (ThreadID tid = 0; tid < numThreads; tid++) {
        status_change = status_change || checkSignalsAndUpdate(tid);
    }

    // 2. 分发
    for (ThreadID tid = 0; tid < numThreads; tid++) {
        dispatch(tid);
    }

    // 3. 执行
    executeInsts();

    // 4. 写回
    writebackInsts();

    // 5. 更新时间缓冲
    issueToExecuteQueue.advance();
    iewQueue->advance();

    // 6. 更新状态
    if (status_change) {
        updateStatus();
    }
}
```

3.2.3 dispatch() 函数

```
void IEW::dispatch(ThreadID tid)
{
    if (dispatchStatus[tid] == Running) {
        dispatchInsts(tid);
    }
}
```

```

    } else if (dispatchStatus[tid] == Unblocking) {
        // 解阻塞
        if (skidBuffer[tid].empty()) {
            DPRINTF(IEW, "[tid:%i] Done unblocking, switching to
running.\n", tid);
            dispatchStatus[tid] = Running;
            toRename->iewInfo[tid].used = true;
            wroteToTimeBuffer = true;
        }
    } else if (dispatchStatus[tid] == Squashing) {
        // Squashing
        if (squashInst[tid] == nullptr) {
            DPRINTF(IEW, "[tid:%i] Done squashing, switching to
running.\n", tid);
            dispatchStatus[tid] = Running;
        }
    }
}
}

```

3.2.4 dispatchInsts() 函数

```

void IEW::dispatchInsts(ThreadID tid)
{
    // 获取可用指令数
    unsigned num_insts = std::min(
        static_cast<unsigned>(insts[tid].size()),
        dispatchWidth);

    for (int i = 0; i < num_insts; i++) {
        DynInstPtr inst = insts[tid].front();
        insts[tid].pop();

        if (inst->isSquashed()) {
            // 指令已被 squash
            DPRINTF(IEW, "[tid:%i] Instruction was squashed, skipping.\n",
tid);

            stats.dispSquashedInsts++;
            continue;
        }

        // 分发到 IQ 或 LSQ
        if (inst->isLoad()) {
            ldstQueue.insertLoad(inst);
            stats.dispLoadInsts++;
        } else if (inst->isStore()) {
            ldstQueue.insertStore(inst);
            stats.dispStoreInsts++;
        } else if (inst->isMemBarrier()) {
            instQueue.insertBarrier(inst);
        } else if (!inst->isNoop()) {
            instQueue.insert(inst);
        }
    }
}

```

```

    }

    if (!inst->isNonSpeculative()) {
        stats.dispNonSpecInsts++;
    }

    DPRINTF(IEW, "[tid:%i] Dispatched instruction [sn:%lli].\n",
            tid, inst->seqNum);
}

stats.dispatchedInsts += num_insts;

// 广播空闲条目数
if (num_insts > 0) {
    updatedQueues = true;
}
}

```

3.2.5 executeInsts() 函数

```

void IEW::executeInsts()
{
    // 从 IQ 获取指令
    DynInstPtr inst;
    while ((inst = instQueue.getInstToExecute())) {
        // 执行指令
        Fault fault = inst->execute();

        if (fault != NoFault) {
            // 执行错误
            DPRINTF(IEW, "Execute fault: %s\n", fault.name());

            if (inst->isAtomic()) {
                // 原子操作错误 - 需要 squash
                squashDueToBranch(inst, inst->threadNumber);
                break;
            } else {
                // 非原子操作 - 直接处理
                DPRINTF(IEW, "[tid:%i] Fault in non-atomic inst: %s\n",
                        inst->threadNumber, fault.name());
            }
        }
    }

    // 检查是否是分支
    if (inst->isControl()) {
        if (inst->predTaken() != inst->traceData.taken ||
            inst->pcState().nextPC() != inst->traceData.branchTarget) {
            // 分支预测错误
            checkMisprediction(inst);
        }
    }
}

```

```

    }

    // 处理 LSQ 执行
    ldstQueue.executeInsts();

    // 检查内存顺序违规
    if (ldstQueue.violation()) {
        // 内存顺序违规 - 需要 squash
        DynInstPtr violator = ldstQueue.getMemDepViolator(
            inst->threadNumber);
        squashDueToMemOrder(violator, violator->threadNumber);
    }
}

```

3.2.6 writebackInsts() 函数

```

void IEW::writebackInsts()
{
    unsigned wb_insts = 0;

    // 写回指令
    while ((wb_insts < wbWidth) &&
        ((fromCommit->commitInfo[tid].doneSeqNum &&
            fromCommit->commitInfo[tid].doneSeqNum > wbNumInst) ||
            toIEW->iewInfo[tid].size > 0)) {

        DynInstPtr inst;

        // 从 IEW 队列获取指令
        if (toIEW->iewInfo[tid].size > 0) {
            inst = toIEW->iewInfo[tid].insts[wbNumInst];
        }

        // 检查指令是否就绪
        if (inst != nullptr && inst->readyToCommit()) {
            // 写回寄存器
            for (int i = 0; i < inst->numDestRegs(); i++) {
                PhysRegIdPtr dest_reg = inst->destReg(i);
                scoreboard->setReg(dest_reg);
            }

            // 唤醒依赖指令
            wakeDependents(inst);

            // 标记指令为已完成
            inst->setCompleted();

            // 发送到提交
            instToCommit(inst);

            wb_insts++;
        }
    }
}

```

```

        } else if (inst != nullptr && !inst->readyToCommit()) {
            // 指令未就绪, 等待
            DPRINTF(IEW, "[tid:%i] Instruction [sn:%lli] not ready to
commit.\n",
                    tid, inst->seqNum);
            break;
        }

        wbNumInst++;
    }

    // 写回统计
    stats.writebackCount.sample(wb_insts);
}

```

3.3 Commit 阶段 - 提交 (commit.hh / commit.cc)

类: **Commit**

3.3.1 Commit 类结构

```

class Commit
{
public:
    enum CommitStatus {
        Active,
        Inactive
    };

    enum ThreadStatus {
        Running,
        Idle,
        ROBSquashing,
        TrapPending,
        FetchTrapPending,
        SquashAfterPending,
        ThreadStatusMax
    };

private:
    CommitStatus _status;
    CommitStatus _nextStatus;
    ThreadStatus commitStatus[MaxThreads];
    CommitPolicy commitPolicy;

    // ROB 指针
    ROB *rob;

    // 线程管理
    std::vector<ThreadState *> thread;

```

```

std::list<ThreadID> *activeThreads;
std::list<ThreadID> priority_list;

// 重命名映射
UnifiedRenameMap *renameMap[MaxThreads];

// PC 状态
std::unique_ptr<PCStateBase> pc[MaxThreads];

// 序列号
InstSeqNum youngestSeqNum[MaxThreads];
InstSeqNum lastCommittedSeqNum[MaxThreads];

// 状态标志
bool wroteToTimeBuffer;
bool changedROBNumEntries[MaxThreads];
bool trapSquash[MaxThreads];
bool tcSquash[MaxThreads];
bool trapInFlight[MaxThreads];
bool committedStores[MaxThreads];
bool checkEmptyROB[MaxThreads];
bool drainPending;
bool drainImminent;
bool canHandleInterrupts;
bool avoidQuiesceLiveLock;

// SquashAfter 支持
DynInstPtr squashAfterInst[MaxThreads];

// HTM 支持
int htmStarts[MaxThreads];
int htmStops[MaxThreads];

// 配置参数
const Cycles iewToCommitDelay;
const Cycles commitToIEWDelay;
const Cycles renameToROBDelay;
const Cycles fetchToCommitDelay;
const unsigned renameWidth;
const unsigned commitWidth;
const Cycles trapLatency;

// 中断
Fault interrupt;
};

```

3.3.2 tick() 函数

```

void Commit::tick()
{
    bool status_change = false;

```

```
// 1. 检查信号并更新状态
for (ThreadID tid = 0; tid < numThreads; tid++) {
    status_change = status_change || checkSignalsAndUpdate(tid);
}

// 2. 提交
commit();

// 3. 更新时间缓冲
iewQueue->advance();
renameQueue->advance();
fetchQueue->advance();

// 4. 更新状态
if (status_change) {
    updateStatus();
}
}
```

3.3.3 commit() 函数

```
void Commit::commit()
{
    // 处理 HTM 统计
    for (ThreadID tid = 0; tid < numThreads; tid++) {
        if (executingHtmTransaction(tid)) {
            commitInsts();
            return;
        }
    }

    // 获取提交线程
    ThreadID tid = getCommittingThread();

    if (tid != -1) {
        // 提交指令
        commitInsts();
    }

    // 获取指令到 ROB
    getInsts();

    // 标记已完成指令
    markCompletedInsts();

    // 处理中断
    handleInterrupt();
}
```

3.3.4 commitInsts() 函数

```

void Commit::commitInsts()
{
    unsigned num_committed = 0;

    // 循环提交指令
    while (num_committed < commitWidth) {
        // 获取提交线程
        ThreadID tid = getCommittingThread();

        if (tid == -1 || commitStatus[tid] != Running) {
            break;
        }

        // 读取 ROB 头部
        DynInstPtr head_inst = rob->readHeadInst(tid);

        if (!head_inst || !head_inst->readyToCommit()) {
            break;
        }

        // 检查是否是分支错误
        if (head_inst->isControl() &&
            (head_inst->misPredicted() ||
             (head_inst->isUncondCtrl() && !head_inst->traceData.taken))) {
            // 分支预测错误
            DPRINTF(Commit, "[tid:%i] Branch misprediction detected.\n",
tid);

            DPRINTF(Commit, "[tid:%i] PC: %#x, Next PC: %#x\n",
                    tid, head_inst->pcState().instAddr(),
                    head_inst->pcState().nextPC());

            // Squash
            youngestSeqNum[tid] = head_inst->seqNum;
            commitStatus[tid] = ROBSquashing;
            toIEW->commitInfo[tid].squash = true;
            toIEW->commitInfo[tid].doneSeqNum = head_inst->seqNum;
            toIEW->commitInfo[tid].pc = &head_inst->pcState();
            toIEW->commitInfo[tid].branchTaken = head_inst-
>traceData.taken;
            toIEW->commitInfo[tid].mispredictInst = head_inst;

            wroteToTimeBuffer = true;

            stats.branchMispredicts++;

            break;
        }

        // 提交指令
        bool committed = commitHead(head_inst, num_committed);
    }
}

```



```

        if (!committed) {
            break;
        }

        num_committed++;
        lastCommittedSeqNum[tid] = head_inst->seqNum;

        // 移除 ROB 头部
        rob->retireHead(tid);

        // 更新统计
        updateComInstStats(head_inst);
    }

    stats.numCommittedDist.sample(num_committed);
}

```

3.3.5 commitHead() 函数

```

bool Commit::commitHead(const DynInstPtr &head_inst, unsigned inst_num)
{
    ThreadID tid = head_inst->threadNumber;
    Fault fault = head_inst->getFault();

    if (fault != NoFault) {
        // 指令有错误
        DPRINTF(Commit, "[tid:%i] Instruction has fault: %s\n", tid,
            fault.name());
        generateTrapEvent(tid, fault);
    };

    if (fault == pageFault() || fault == alignmentFault()) {
        // 页面错误或对齐错误
        toIEW->commitInfo[tid].doneSeqNum = head_inst->seqNum;
        toIEW->commitInfo[tid].squash = true;
        toIEW->commitInfo[tid].pc = &head_inst->pcState();
        toIEW->commitInfo[tid].branchTaken = false;
        toIEW->commitInfo[tid].mispredictInst = head_inst;

        wroteToTimeBuffer = true;
        return false;
    }

    return false;
}

// 清理重命名历史
for (int i = 0; i < (*head_inst).numDestRegs(); i++) {
    renameMap[tid]->setEntry((*head_inst).destRegIdx(i),
        (*head_inst).renameDestReg(i));
}

```

```

        freeList->addReg((*head_inst).destRegIdx(i).classType(),
                        (*head_inst).renameDestReg(i));
        stats.committedMaps++;
    }

    // 标记指令为已提交
    head_inst->setCommitted();

    // 更新 PC
    pc[tid] = *head_inst;

    DPRINTF(Commit, "[tid:%i] Committed instruction [sn:%lli], PC %#x\n",
            tid, head_inst->seqNum, pc[tid]->instAddr());

    return true;
}

```

3.4 IQ - 指令队列 (inst_queue.hh / inst_queue.cc)

类: **InstructionQueue**

3.4.1 IQ 类结构

```

class InstructionQueue
{
public:
    // FU 完成事件
    class FUCompletion : public Event
    {
    private:
        DynInstPtr inst;
        FUPool *fuPool;
        int fuIdx;
        InstructionQueue *iqPtr;
        bool freeFU;
    public:
        FUCompletion(const DynInstPtr &_inst, FUPool *fu_pool, int fu_idx,
                    InstructionQueue *iq_ptr);
        virtual void process();
        void setFreeFU() { freeFU = true; }
    };

private:
    // 指令列表
    std::list<DynInstPtr> instList[MaxThreads];

    // 就绪指令
    std::list<DynInstPtr> instsToExecute;

    // 延迟内存指令

```

```

std::list<DynInstPtr> deferredMemInsts;
std::list<DynInstPtr> blockedMemInsts;
std::list<DynInstPtr> retryMemInsts;

// 优先级队列
struct PqCompare {
    bool operator()(const DynInstPtr &lhs, const DynInstPtr &rhs)
const;
};

typedef std::priority_queue<
    DynInstPtr, std::vector<DynInstPtr>, PqCompare> ReadyInstQueue;

ReadyInstQueue readyInsts[Num_OpClasses];

// 非推测指令
std::map<InstSeqNum, DynInstPtr> nonSpecInsts;

// 依赖图
DependencyGraph<DynInstPtr> dependGraph;

// 内存依赖单元
MemDepUnit memDepUnit[MaxThreads];

// 年龄顺序列表
struct ListOrderEntry {
    OpClass queueType;
    InstSeqNum oldestInst;
};
std::list<ListOrderEntry> listOrder;
ListOrderIt readyIt[Num_OpClasses];
bool queueOnList[Num_OpClasses];

// 配置参数
ThreadID numThreads;
unsigned totalWidth;
unsigned numPhysRegs;
int wbOutstanding;
Cycles commitToIEWDelay;
InstSeqNum squashedSeqNum[MaxThreads];
std::vector<bool> regScoreboard;
};

```

3.4.2 insert() 函数

```

void InstructionQueue::insert(const DynInstPtr &new_inst)
{
    ThreadID tid = new_inst->threadNumber;
    OpClass op_class = new_inst->opClass();

    // 添加到指令列表

```

```
    instList[tid].push_back(new_inst);

    // 设置指令状态
    new_inst->setInIQ();

    // 添加到依赖图
    addToDependents(new_inst);
    addToProducers(new_inst);

    // 检查是否就绪
    addIfReady(new_inst);

    DPRINTF(IQ, "[tid:%i] Instruction added to IQ.\n", tid);
    stats.instsAdded++;
}
```

3.4.3 scheduleReadyInsts() 函数

```
void InstructionQueue::scheduleReadyInsts()
{
    unsigned to_execute = 0;

    // 循环调度指令
    while (to_execute < totalWidth) {
        DynInstPtr inst = nullptr;

        // 检查年龄顺序列表
        if (!listOrder.empty()) {
            ListOrderEntry entry = listOrder.front();
            inst = getInstToExecute(entry.queueType);

            if (inst == nullptr) {
                moveToYoungerInst(listOrder.begin());
                continue;
            }

            listOrder.pop_front();
        } else {
            break;
        }

        // 获取功能单元
        FUPool *fu_pool = findFU(inst);

        if (fu_pool == nullptr) {
            // 没有可用的功能单元
            stats.fuBusy[inst->opClass()]++;
            break;
        }

        int fu_idx = fu_pool->getUnit(inst->opClass());
```

```

        if (fu_idx == -1) {
            // 功能单元忙
            stats.fuBusy[inst->opClass()]++;
            break;
        }

        // 设置指令为已发射
        inst->setIssued();

        // 创建 FU 完成事件
        Cycles op_lat = fu_pool->getOpLatency(inst->opClass());

        if (op_lat == 1) {
            // 无延迟指令
            processFUCompletion(inst, fu_pool, fu_idx);
        } else {
            // 有延迟指令 - 调度事件
            FUCompletion *event = new FUCompletion(inst, fu_pool, fu_idx,
this);
            schedule(event, clockEdge(op_lat - 1));
        }

        to_execute++;
    }

    stats.numIssuedDist.sample(to_execute);
}

```

3.5 LSQ - 加载存储队列 (lsq.hh / lsq.cc)

类: **LSQ**

3.5.1 LSQ 类结构

```

class LSQ
{
public:
    // LSQ 请求类
    class LSQRequest : public BaseMMU::Translation, public
Packet::SenderState
    {
public:
        enum class State {
            NotIssued,
            Translation,
            Request,
            Fault,
            PartialFault,
        };
    };
};

```

```

enum Flag : FlagsStorage {
    IsLoad                = 0x00000001,
    WriteBackToRegister   = 0x00000002,
    Delayed                = 0x00000004,
    IsSplit                = 0x00000008,
    TranslationStarted     = 0x00000010,
    TranslationFinished    = 0x00000020,
    Sent                  = 0x00000040,
    Retry                 = 0x00000080,
    Complete              = 0x00000100,
    TranslationSquashed    = 0x00000200,
    Discarded              = 0x00000400,
    LSQEntryFreed         = 0x00000800,
    WritebackScheduled     = 0x00001000,
    WritebackDone          = 0x00002000,
    IsAtomic               = 0x00004000
};

protected:
    FlagsType flags;
    State _state;

public:
    LSQUnit& _port;
    const DynInstPtr _inst;
    uint32_t _taskId;
    PacketDataPtr _data;
    std::vector<PacketPtr> _packets;
    std::vector<RequestPtr> _reqs;
    std::vector<Fault> _fault;
    uint64_t* _res;
    const Addr _addr;
    const uint32_t _size;
    const Request::Flags _flags;
    std::vector<bool> _byteEnable;
    uint32_t _numOutstandingPackets;
    AtomicOpFunctorPtr _amo_op;
    bool _hasStaleTranslation;

    // 方法
    LSQRequest(LSQUnit* port, const DynInstPtr& inst, bool isLoad);
    LSQRequest(LSQUnit* port, const DynInstPtr& inst, bool isLoad,
               const Addr& addr, const uint32_t& size,
               const Request::Flags& flags_, PacketDataPtr data=nullptr,
               uint64_t* res=nullptr, AtomicOpFunctorPtr amo_op=nullptr,
               bool stale_translation=false);

    void install();
    void release(Flag reason);
    bool squashed() const override;
    void addReq(Addr addr, unsigned size,
               const std::vector<bool>& byte_enable);
    virtual ~LSQRequest();

```

```

    void initiateTranslation() = 0;
    void finish(const Fault &fault, const RequestPtr &req,
               gem5::ThreadContext* tc, BaseMMU::Mode mode);
    void buildPackets();
    bool recvTimingResp(PacketPtr pkt);
    void sendPacketToCache();
};

class SingleDataRequest : public LSQRequest
{
public:
    SingleDataRequest(LSQUnit* port, const DynInstPtr& inst,
                     bool isLoad, const Addr& addr, const uint32_t& size,
                     const Request::Flags& flags_, PacketDataPtr data=nullptr,
                     uint64_t* res=nullptr, AtomicOpFunctorPtr amo_op=nullptr);
    virtual void initiateTranslation();
    virtual bool recvTimingResp(PacketPtr pkt);
    virtual void sendPacketToCache();
    virtual void buildPackets();
};

class SplitDataRequest : public LSQRequest
{
public:
    SplitDataRequest(LSQUnit* port, const DynInstPtr& inst,
                     bool isLoad, const Addr& addr, const uint32_t& size,
                     const Request::Flags & flags_, PacketDataPtr data=nullptr,
                     uint64_t* res=nullptr);
    virtual bool recvTimingResp(PacketPtr pkt);
    virtual void initiateTranslation();
    virtual void sendPacketToCache();
    virtual void buildPackets();
};

private:
    // D-cache 端口
    class DcachePort : public RequestPort
    {
    protected:
        LSQ *_lsq;
        CPU *_cpu;
    public:
        DcachePort(LSQ *_lsq, CPU *_cpu);
        virtual bool recvTimingResp(PacketPtr pkt);
        virtual void recvTimingSnoopReq(PacketPtr pkt);
        virtual void recvReqRetry();
        virtual bool isSnooping() const { return true; }
        bool throttleReadResp(PacketPtr pkt);
    };

    // LSQ 单元
    std::vector<LSQUnit> thread;

    // 配置

```

```

    unsigned LQEntries;
    unsigned SQEntries;
    unsigned maxLQEntries;
    unsigned maxSQEntries;
    DcachePort dcachePort;
    SMTQueuePolicy lsqPolicy;
    const bool recvRespThrottling;
    const unsigned recvRespMaxCachelines;
    const unsigned recvRespBufferSize;
};

```

3.5.2 LSQUnit - 每线程 LSQ 单元

```

class LSQUnit
{
public:
    // LSQ 条目
    struct LSQEntry
    {
        DynInstPtr inst;
        LSQRequest *request;
        bool valid;
        bool pending;
        bool load;
        bool store;
        bool issued;
        bool committed;
        uint8_t size;
        Addr addr;
        uint8_t data[MaxDataWidth];
    };

    struct LQEntry : public LSQEntry
    {
        // 加载队列条目
    };

    struct SQEntry : public LSQEntry
    {
        // 存储队列条目
    };

private:
    std::deque<LQEntry> loadQueue;
    std::deque<SQEntry> storeQueue;

    // 配置
    unsigned lqSize;
    unsigned sqSize;
    bool cacheBlocked;
    int storeOffset;

```



```
int loadOffset;

// 状态标志
bool storeWaitingOnWB;
bool storeToWB;
bool loadWaitingOnWB;
bool loadBlockedOnStore;
Addr lastStoreAddr;
uint32_t lastStoreSize;

// 内存依赖
MemDepEntry *memDepEntry;
DynInstPtr memDepViolator;
};
```

3.6 ROB - 重排序缓冲 (rob.hh / rob.cc)

类: **ROB**

3.6.1 ROB 类结构

```
class ROB
{
public:
    enum Status {
        Running,
        Idle,
        ROBSquashing
    };

private:
    Status robStatus[MaxThreads];
    SMTQueuePolicy robPolicy;

    // 指令列表
    std::list<DynInstPtr> instList[MaxThreads];

    // 资源管理
    unsigned numEntries;
    unsigned threadEntries[MaxThreads];
    unsigned maxEntries[MaxThreads];

    // 迭代器
    InstIt tail;
    InstIt head;
    InstIt squashIt[MaxThreads];

    // Squash 状态
    InstSeqNum squashedSeqNum[MaxThreads];
    bool doneSquashing[MaxThreads];
};
```

```
    // Squash 宽度
    const std::optional<unsigned> squashWidth;
};
```

3.6.2 insertInst() 函数

```
void ROB::insertInst(const DynInstPtr &inst)
{
    ThreadID tid = inst->threadNumber;

    // 检查 ROB 是否满
    if (isFull(tid)) {
        panic("ROB full");
    }

    // 插入指令
    instList[tid].push_back(inst);

    // 更新尾迭代器
    tail = instList[tid].end();
    tail--;

    // 更新计数
    threadEntries[tid]++;
    numInstsInROB++;

    // 更新 ROB 宽度
    if (numInstsInROB > numEntries) {
        numEntries = numInstsInROB;
    }

    DPRINTF(ROB, "[tid:%i] Instruction [sn:%lli] inserted to ROB.\n",
            tid, inst->seqNum);
}
```

3.6.3 squash() 函数

```
void ROB::squash(InstSeqNum squash_num, ThreadID tid)
{
    DPRINTF(ROB, "[tid:%i] Squashing until sequence number %i.\n",
            tid, squash_num);

    // 设置 squash 序列号
    squashedSeqNum[tid] = squash_num;

    // 循环 squash 指令
    int squash_count = 0;
```

```
bool done_squashing = true;

for (InstIt it = instList[tid].begin(); it != instList[tid].end();
++it) {
    if ((*it)->seqNum > squash_num) {
        // 标记为已 squash
        (*it)->setSquashed();
        squash_count++;

        if (squashWidth) {
            if (squash_count >= *squashWidth) {
                done_squashing = false;
                break;
            }
        }
    }
}

if (done_squashing) {
    doneSquashing[tid] = true;
}
}
```

四、支持结构

4.1 寄存器管理

4.1.1 PhysRegFile - 物理寄存器文件

```
class PhysRegFile
{
public:
    // 构造函数
    PhysRegFile(unsigned _numPhysIntRegs,
                unsigned _numPhysFloatRegs,
                unsigned _numPhysVecRegs,
                unsigned _numPhysVecPredRegs,
                unsigned _numPhysMatRegs,
                unsigned _numPhysCCRegs,
                const std::vector<RegClass> &_regClasses)
        : regClasses(_regClasses)
    {
        // 初始化各类寄存器
        initRegs(IntRegClass, _numPhysIntRegs);
        initRegs(FloatRegClass, _numPhysFloatRegs);
        initRegs(VecRegClass, _numPhysVecRegs);
        initRegs(VecPredRegClass, _numPhysVecPredRegs);
        initRegs(MatRegClass, _numPhysMatRegs);
        initRegs(CCRegClass, _numPhysCCRegs);
    }
};
```

```

    }

    // 寄存器访问
    RegVal getReg(PhysRegIdPtr phys_reg);
    void getReg(PhysRegIdPtr phys_reg, void *val);
    void *getWritableReg(PhysRegIdPtr phys_reg);
    void setReg(PhysRegIdPtr phys_reg, RegVal val);
    void setReg(PhysRegIdPtr phys_reg, const void *val);

private:
    std::vector<RegClass> regClasses;
    std::vector<std::vector<uint8_t>> regs;
};

```

4.1.2 UnifiedFreeList - 统一空闲列表

```

class UnifiedFreeList
{
public:
    // 每寄存器类空闲列表
    class SimpleFreeList
    {
    public:
        std::list<PhysRegIdPtr> freeList;
        unsigned numFreeRegs;

        PhysRegIdPtr getReg();
        void addReg(PhysRegIdPtr free_reg);
        void addRegs(std::list<PhysRegIdPtr> &free_regs);
        void reset();
    };

    // 构造函数
    UnifiedFreeList(const std::string &_name, PhysRegFile *_regFile)
        : regFile(_regFile)
    {
        // 初始化各类空闲列表
        for (int i = 0; i < Num_RegClassType; i++) {
            freeLists[i] = new SimpleFreeList();
        }
    }

    // 空理寄存器分配
    PhysRegIdPtr getReg(RegClassType type);

    // 空理寄存器释放
    void addReg(RegClassType type, PhysRegIdPtr free_reg);
    void addRegs(RegClassType type, std::list<PhysRegIdPtr> &free_regs);

    // 空闲寄存器计数
    unsigned numFreeRegs();

```

```

        unsigned numFreeRegs(RegClassType type);

private:
    PhysRegFile *regFile;
    SimpleFreeList *freeLists[Num_RegClassType];
};

```

4.1.3 UnifiedRenameMap - 统一重命名映射

```

class UnifiedRenameMap
{
public:
    // 每寄存器类映射
    class SimpleRenameMap
    {
    public:
        std::vector<PhysRegIdPtr> map;

        PhysRegIdPtr lookup(const RegId &arch_reg);
        void setEntry(const RegId &arch_reg, PhysRegIdPtr phys_reg);
        void init(std::vector<RegClass> &regClasses,
                  PhysRegFile *regFile,
                  UnifiedFreeList *freeList);
    };

    // 构造函数
    UnifiedRenameMap()
    {
        for (int i = 0; i < Num_RegClassType; i++) {
            maps[i] = new SimpleRenameMap();
        }
    }

    // 映射表操作
    PhysRegIdPtr rename(const RegId &arch_reg, PhysRegFile *regFile,
                        UnifiedFreeList *freeList);
    PhysRegIdPtr lookup(const RegId &arch_reg);
    void setEntry(const RegId &arch_reg, PhysRegIdPtr phys_reg);

    // 初始化
    void init(std::vector<RegClass> &regClasses,
              PhysRegFile *regFile,
              UnifiedFreeList *freeList);

    // 空闲条目数
    unsigned minFreeEntries();

private:
    SimpleRenameMap *maps[Num_RegClassType];
};

```

4.1.4 Scoreboard - 记分板

```
class Scoreboard
{
public:
    Scoreboard(const std::string &_name, unsigned _numPhysRegs)
        : name(_name), numPhysRegs(_numPhysRegs)
    {
        // 初始化记分板
        regScoreboard.resize((numPhysRegs + 8) / 8, 0);
    }

    // 检查寄存器
    bool getReg(PhysRegIdPtr phys_reg) const
    {
        unsigned reg_idx = phys_reg->index();
        return regScoreboard[reg_idx / 8] & (1 << (reg_idx % 8));
    }

    // 设置寄存器
    void setReg(PhysRegIdPtr phys_reg)
    {
        unsigned reg_idx = phys_reg->index();
        regScoreboard[reg_idx / 8] |= (1 << (reg_idx % 8));
    }

    // 清除寄存器
    void unsetReg(PhysRegIdPtr phys_reg)
    {
        unsigned reg_idx = phys_reg->index();
        regScoreboard[reg_idx / 8] &= ~(1 << (reg_idx % 8));
    }

private:
    std::string name;
    unsigned numPhysRegs;
    std::vector<uint8_t> regScoreboard;
};
```

4.2 内存依赖

4.2.1 MemDepUnit - 内存依赖单元

```
class MemDepUnit
{
public:
    // 内存依赖条目
    struct MemDepEntry
    {
        /
```

```

        InstSeqNum instSeqNum;
        DynInstPtr inst;
        MemDepEntry *next;
        MemDepEntry *prev;
        bool memBarrier;
        bool hit;
};

// 构造函数
MemDepUnit()
{
    // 初始化 store set
    storeSet.init();
}

// 插入指令
void void insert(const DynInst::MicroOp &new_inst);
void insertNonSpec(const DynInst::MicroOp &new_inst);
void insertBarrier(const DynInst::MicroOp &barr_inst);

// 指令完成
void completeInst(const DynInst::MicroOp &inst);
void nonSpecInstReady(const DynInst::MicroOp &inst);

// 内存顺序违规
void violation(const DynInst::MicroOp &store, const DynInst::MicroOp
&load);

// 重放
void reschedule(const DynInst::MicroOp &inst);
void replay(const DynInst::MicroOp &inst);

// Squash
void squash(const InstSeqNum &squash_inst, ThreadID tid);

// 唤醒依赖
void wakeDependents(const DynInst::MicroOp &inst);

private:
    std::list<MemDepEntry> memDepList;
    std::map<InstSeqNum, DynInstPtr> memInsts;
    StoreSet storeSet;
};

```

4.2.2 StoreSet - Store Set 预测器

```

class StoreSet
{
public:
    // SSIT 条目
    struct SSITEntry

```

```
{
    Addr tag;
    int64_t lastIndex;
    int64_t currentIndex;
    uint8_t valid;
};

// LFST 条目
struct LFSTEntry
{
    Addr PC;
    int64_t index;
    uint8_t valid;
};

// 构造函数
StoreSet(int SSITSize, int LFSTSize)
    : ssitSize(SSITSize), lfstSize(LFSTSize)
{
    // 初始化 SSIT 和 LFST
    SSIT.resize(ssitSize);
    LFST.resize(lfstSize);
}

// 记录违规
void violation(Addr load_PC, Addr store_PC, int store_idx, ThreadID
tid);

// 插入指令
void insertLoad(const DynInstPtr &inst, ThreadID tid);
void insertStore(const DynInstPtr &inst, ThreadID tid);

// 检查依赖
void checkInst(const DynInstPtr &inst, ThreadID tid);

// 发射
void issued(const DynInstPtr &inst, ThreadID tid);

// Squash
void squash(InstSeqNum squashed_inst, ThreadID tid);

private:
    int ssitSize;
    int lfstSize;
    std::vector<SSITEntry> SSIT;
    std::vector<LFSTEntry> LFST;
};
```

4.2.3 DependencyGraph - 依赖图


```

template <class T>
class DependencyGraph
{
public:
    // 依赖条目
    struct DependencyEntry
    {
        T node;
        DependencyEntry *next;
    };

    // 构造函数
    DependencyGraph(unsigned _numPhysRegs)
        : numPhysRegs(_numPhysRegs)
    {
        // 初始化依赖图
        dependGraph.resize((numPhysRegs + 8) / 8, nullptr);
    }

    // 插入依赖
    void insert(const T &inst, bool track_producers);

    // 设置生产者
    void setInst(const TRegId &reg, const T &inst);

    // 清除生产者
    void clearInst(const TRegId &reg);

    // 移除指令
    void remove(const T &inst);

    // 弹出最新依赖
    T pop();

private:
    unsigned numPhysRegs;
    std::vector<DependencyEntry *> dependGraph;
};

```

4.3 功能单元

4.3.1 FUPool - 功能单元池

```

class FUPool
{
public:
    // FU 索引队列
    class FUIdxQueue
    {

```

```

    public:
        std::deque<int> freeList;
        int capacity;

        FUIdxQueue(int size) : capacity(size) {}

        int getUnit();
        void freeUnitNextCycle(int idx);
        void processFreeUnits();
};

// FU 链源描述
struct FUDesc
{
    std::string name;
    int count;
    unsigned opLat;
    OpClass opClass;
};

// 构造函数
FUPool(FUDesc *desc, int num);

// 获取 FU
int getUnit(OpClass op_class);

// 释放 FU
void freeUnitNextCycle(int idx);
void processFreeUnits();

// 检查能力
bool isCapable(OpClass op_class);

private:
    std::string name;
    std::vector<int> fuIndices;
    std::vector<bool> busy;
    std::vector<unsigned> opLatencies;
    std::vector<OpClass> opClasses;
    std::vector<FUIdxQueue> fuQueues;
};

```

4.4 线程管理

4.4.1 ThreadState - 线程状态

```

class ThreadState : public gem5::ThreadState
{
    public:
        // 构造函数

```

```

ThreadState(BaseCPU *cpu, ThreadID _tid, Process *_process,
             ContextID _context_id = 0);

// 线程控制
void activate();
void suspend();
void halt();

// 状态查询
bool status() const { return _status; }

// 事件队列
gem5::PCEventQueue pcEventQueue() { return _pcEventQueue; }

// 中断
void trap(const Fault &fault);

private:
    Status _status;
    Process *process;
    ThreadID _tid;
    ContextID _contextId;
    gem5::PCEventQueue *_pcEventQueue;
};

```

4.4.2 ThreadContext - 线程上下文

```

class ThreadContext : public gem5::ThreadContext
{
public:
    // 寄存器访问
    RegVal readMiscRegNoEffect(int misc_reg) const;
    RegVal readMiscReg(int misc_reg);
    void setMiscRegNoEffect(int misc_reg, RegVal val);
    void setMiscReg(int misc_reg, RegVal val);

    // 架构寄存器访问
    RegVal readIntReg(int reg_idx) const;
    RegVal readFloatReg(int reg_idx) const;
    RegVal readVecReg(int reg_idx) const;
    RegVal readVecElemReg(int reg_idx, int elem_idx) const;
    void setIntReg(int reg_idx, RegVal val);
    void setFloatReg(int reg_idx, RegVal val);
    void setVecReg(int reg_idx, RegVal val);
    void setVecElemReg(int reg_idx, int elem_idx, RegVal val);

    // PC 访问
    const PCStateBase &pcState() const;
    void pcState(const PCStateBase &new_pc_state);

    // 线程控制

```

```

    void activate();
    void suspend();
    void halt();

    // 统计
    ContextId contextId();

private:
    O3CPU *cpu;
    ThreadState *thread;
};

```

五、动态指令

5.1 DynInst - 动态指令 (dyn_inst.hh / dyn_inst.cc)

类: **DynInst**

5.1.1 DynInst 类结构

```

class DynInst : public ExecContext, public RefCounted
{
public:
    // 状态标志
    union
    {
        bool status_bits[32];
        struct
        {
            bool readyToIssue : 1;
            bool issued : 1;
            bool canCommit : 1;
            bool readyToCommit : 1;
            bool executed : 1;
            bool completed : 1;
            bool canWB : 1;
            bool squashed : 1;
            bool staticInst : 1;
            bool traceData : 1;
            bool miscReady : 1;
            bool eaSrc : 1;
            bool eaDest : 1;
            bool predicate : 1;
            bool memOp : 1;
            bool moveForward : 1;
            bool splitRequest : 1;
            bool atCommit : 1;
            bool inIQ : 1;
            bool inLSQ : 1;
            bool inROB : 1;

```

```

    };
} status;

// 执行上下文
TheISA::PCState _pcState;
TheISA::PCState _predPC;
TheISA::PCState _nextPC;
std::unique_ptr<PCStateBase> _pc;
std::unique_ptr<PCStateBase> _next_pc;

// 指令信息
ThreadID _threadNumber;
InstSeqNum _seqNum;
StaticInstPtr _staticInst;
StaticInstPtr _macroop;

// 寄存器映射
RegId _srcRegIdx[MaxInstSrcRegs];
PhysRegIdPtr _srcRegIdxRename[MaxInstSrcRegs];
RegId _destRegIdx[MaxInstDestRegs];
PhysRegIdPtr _renameDestraIdx[MaxInstDestRegs];
PhysRegIdPtr _renameSrcRegIdx[MaxInstSrcRegs];
PhysRegIdPtr _renameDestRegIdx[MaxInstDestRegs];

// 执行结果
union
{
    uint64_t iresult;
    double fresult;
} _result;

// 跟踪数据
Trace::InstRecord *traceData;

public:
    // 构造函数
    DynInst(StaticInstPtr _staticInst, StaticInstPtr _macroop,
            DynamicInstInfo *dynamic_info, InstSeqNum seq_num,
            ThreadID tid);
    virtual ~DynInst();

    // 执行
    Fault execute();

    // 寄存器访问
    RegVal getRegOperand(const StaticInst *si, int idx);
    void setRegOperand(const StaticInst *si, int idx, RegVal val);

    // 状态设置
    void setReadyToIssue() { status.readyToIssue = true; }
    void setIssued() { status.issued = true; }
    void setCanCommit() { status.canCommit = true; }
    void setReadyToCommit() { status.readyToCommit = true; }
    void setExecuted() { status.executed = true; }

```

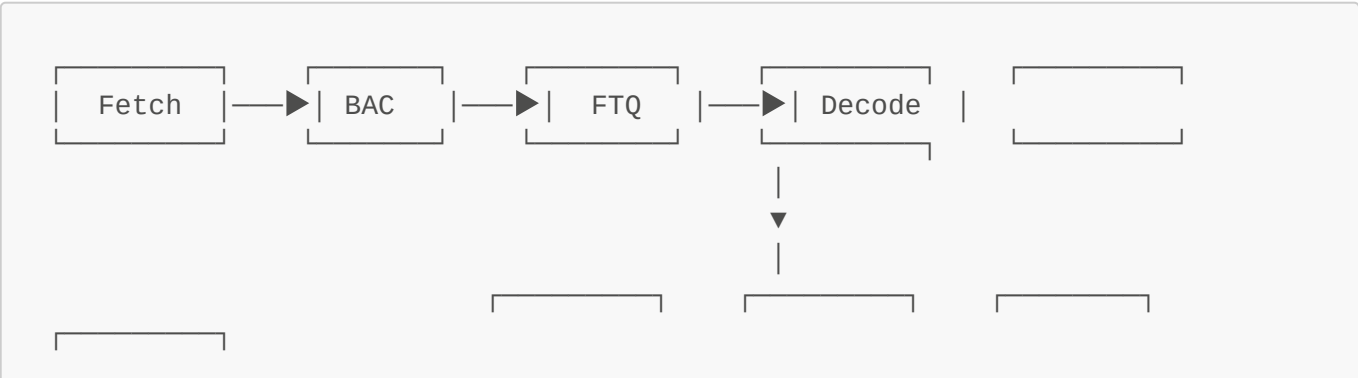
```
void setCompleted() { status.completed = true; }
void setCanWB() { status.canWB = true; }
void setSquashed() { status.squashed = true; }
void setInIQ() { status.inIQ = true; }
void setInLSQ() { status.inLSQ = true; }
void setInROB() { status.inROB = true; }

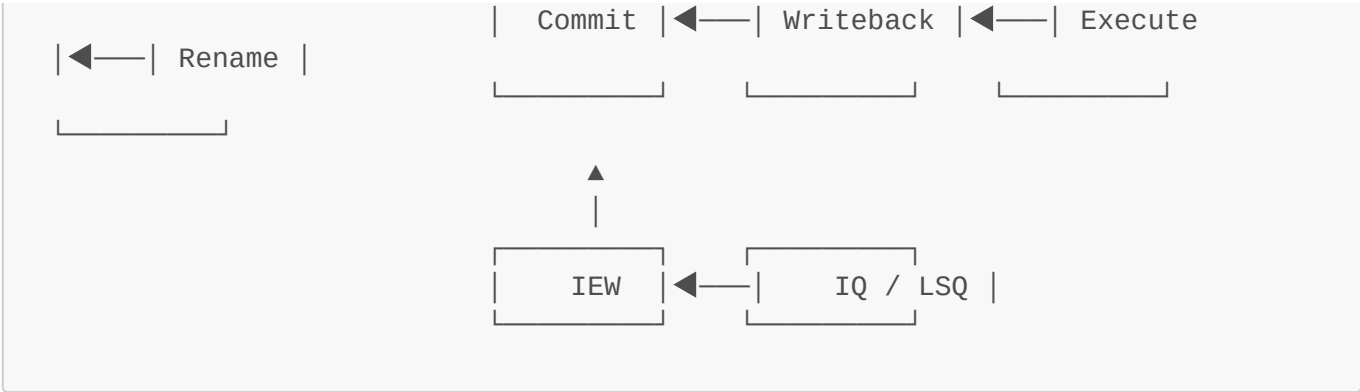
// 状态查询
bool readyToIssue() const { return status.readyToIssue; }
bool isIssued() const { return status.issued; }
bool canCommit() const { return status.canCommit; }
bool readyToCommit() const { return status.readyToCommit; }
bool isExecuted() const { return status.executed; }
bool isCompleted() const { return status.completed; }
bool canWB() const { return status.canWB; }
bool isSquashed() const { return status.squashed; }
bool isInIQ() const { return status.inIQ; }
bool isInLSQ() const { return status.inLSQ; }
bool isInROB() const { return status.inROB; }

// 指令类型查询
bool isLoad() const;
bool isStore() const;
bool isControl() const;
bool isMemRef() const;
bool isAtomic() const;
bool isMemBarrier() const;
bool isQuiesce() const;
bool isSerializeBefore() const;
bool isSerializeAfter() const;
bool isNoop() const;
bool isMicroop() const;
bool isMacroop() const;
bool isFirstMicroop() const;
bool isLastMicroop() const;
bool isVector() const;
};
```

六、总结

6.1 流水线数据流图





6.2 关键特性

特性	描述
SMT 支持	最多 4 线程同时多线程
可配置共享策略	IQ/LSQ/ROB 的线程间分区策略
内存依赖预测	基于 store sets 的详细预测器
解耦前端	BAC/FTQ 分离分支预测和取指
乱序执行	完整的寄存器重命名和依赖跟踪
精确异常	通过 ROB 实现顺序提交

6.3 文件列表

文件	功能
cpu.hh/cc	主 CPU 类，协调所有阶段
fetch.hh/cc	取指阶段
bac.hh/cc	分支地址计算阶段
ftq.hh/cc	取指目标队列
decode.hh/cc	译码阶段
rename.hh/cc	重命名阶段
iew.hh/cc	发射/执行/写回阶段
commit.hh/cc	提交阶段
inst_queue.hh/cc	指令队列
lsq.hh/cc	加载存储队列
rob.hh/cc	重排序缓冲
regfile.hh	物理寄存器文件
free_list.hh	空理寄存器空闲列表
rename_map.hh	重命名映射

文件	功能
scoreboard.hh	记分板
mem_dep_unit.hh/cc	内存依赖单元
store_set.hh/cc	Store set 预测器
dep_graph.hh	依赖图
fu_pool.hh/cc	功能单元池
thread_context.hh/cc	线程上下文
thread_state.hh/cc	线程状态
comm.hh	阶段间通信结构
dyn_inst.hh/cc	动态指令

生成信息

- 生成时间: 2026-04-16
- 分析的目录: /home/mikasa/0403/gem5/src/cpu/o3
- gem5 版本: v25.1.0.0